CRIMEVISIONAI

Solution Architecture

Folder 3 | Logical architecture | Prepared 23 October 2026

Project: CrimeVisionAI — Multimodal Forensic Evidence Analysis and Investigation Support System

Document status: Project-specific draft based on the implementation information currently available. Verify placeholders and run-dependent results before formal submission.

1. Architecture diagram (logical)

Presentation / Client Layer
Browser dashboard: HTML + CSS + vanilla JavaScript + Jinja2 templates
↕ HTTP requests / rendered pages / JSON responses
Application Layer — Flask 3.x app factory
Case workflow | Evidence upload & media | Audio analysis | Image analysis | Video analysis | Timeline & case chat
↕
Processing / Integration Layer
Whisper base | RT-DETR | CLIP | BLIP | Groq API (openai/gpt-oss-20b)
↕
Data Layer
SQLite (cases, evidence, analysis_results) + local evidence file storage

2. Component responsibilities

Component	Responsibility	Technology
Presentation	Case/evidence views, uploads, results, timeline and Q&A UI	Jinja2, HTML, CSS, JavaScript
HTTP/API layer	Validate requests, route work and return JSON/media	Flask 3.x
Case/evidence service	Maintain case and evidence metadata; associate analysis with evidence	Python + sqlite3
Audio pipeline	Transcribe and produce timestamped speech text; structure candidate events	Whisper base + Groq
Image pipeline	Detect objects, rank scene labels, caption image and produce cautious interpretation	RT-DETR + CLIP + BLIP + Groq
Video pipeline	Sample frames and aggregate frame-level visual analysis	OpenCV/video processing as present in implementation; confirm exact library/version in source
Persistence	Store case/evidence/analysis records and uploaded files	SQLite + local filesystem
External LLM integration	Generate structured interpretations and answer questions from available context	Groq API

3. Data model

Reported relational tables: cases, evidence and analysis_results. A case can have multiple evidence records; each evidence item can have associated analysis results. The exact column definitions, foreign-key constraints and deletion behavior must be verified from the current database schema before a formal ER diagram is finalized.

4. Deployment topology and security

The known runtime is local development via run.py at 127.0.0.1:5000 with debug=True. This is not a production deployment diagram. Before shared deployment, disable debug mode, configure secret management, authentication and authorization, upload scanning and retention rules, HTTPS, audit logging, resource limits and isolated processing.

5. Key architectural risks

Local ML inference can be resource-intensive; synchronous requests may block under load. SQLite/local disk are simple but not ideal for concurrent multi-user deployments. External LLM calls introduce network, cost, availability and privacy considerations. Q&A; must be grounded in stored evidence and cite the source record/timestamp where possible.